TRACE: A Tsetlin Runtime Assurance and Certification Engine for Interpretable, Tamper-Evident Guardrails on Agentic AI

Anwar Debes
University of Agder
Draft v1.0, June 2026

Abstract

Large language model (LLM) agents now take consequential actions (reading records, sending messages, moving money), yet the guardrails deployed to keep them safe are, almost without exception, multi-billion-parameter neural classifiers: accurate but opaque, GPU-bound, latency-unspecified, retrained at data-centre cost, and, critically, producing no auditable record an external party can verify. None of the eight production guardrails I survey exposes a tamper-evident audit trail, and only one performs any formal reasoning (and that over single facts, not behaviour over time). I present **TRACE**, a Tsetlin Runtime Assurance and Certification Engine: a CPU-only, interpretable, sub-millisecond (at its deployed clause budget) guardrail that wraps any agent and emits, for every session, a *tamper-evident, independently-verifiable certificate* of what it checked. TRACE has three components on one logic substrate, the Echo-Trace Tsetlin Automaton of THGTM [Debes, 2026]: (i) a *content guard*, a Tsetlin Machine whose learned clauses are human-readable conjunctions of tokens; (ii) a *behaviour guard*, a bank of online finite-trace-LTL monitors whose memory is constant in the trajectory length and whose per-step cost is $O(1)$; and (iii) a *certificate ledger*, a hash-chained, HMAC-signed, Merkle-rooted log that an auditor re-verifies without the model. On two real Apache-2.0 prompt-injection datasets the content guard reaches 0.827 ± 0.036 F1, *significantly above* the strongest CPU baseline (*LinearSVC* 0.800, Mann-Whitney $p = 0.028$), at a 107 μs median (176 μs mean) per check and 12.5 KiB. On a four-family trajectory benchmark the memoryless per-call paradigm is caught in a bind: it either misses the attack entirely (slow-roll exfiltration: attack success rate 1.0) or can only catch it by flagging every benign session (missing-audit and privilege: false-positive rate 1.0), while TRACE's online monitors achieve attack success rate 0 at false-positive rate ≤ 0.05 on all four. Six tamper classes against the ledger are detected at rate 1.00 and localised. I give a precise account of the guarantees (tamper-evidence and symmetric authentication, not non-repudiation), an honest catalogue of failure modes (distribution shift), and I position TRACE against both neural guardrails and the 2025 agent-enforcement line. All numbers, figures and certificates regenerate from the repository on a single CPU.

1 Introduction

The deployment surface of LLM agents has moved, in under two years, from chatbots that emit text to autonomous systems that call tools, browse, write code, and act on a user's behalf. With agency comes a safety problem that single-turn content moderation does not capture: an agent is unsafe *even without an adversary* (the best undefended GPT-4o agent in AgentDojo [Debenedetti et al., 2024] succumbs to a targeted prompt-injection 47.7% of the time on the "important message" attack), and the most damaging attacks unfold *over a trajectory*, not in a single call: indirect prompt injection [Greshake et al., 2023], slow-roll data exfiltration, out-of-role tool use,

and privilege escalation are all behaviours the OWASP Top 10 for Agentic Applications [OWASP Agentic Security Initiative, 2025] codifies.

At the same time, the regulatory floor is rising. The EU AI Act [European Parliament and Council, 2024], whose high-risk obligations apply from 2 August 2026, requires high-risk systems to *automatically log events over their lifetime* (Article 12), to remain robust to data and model poisoning and adversarial evasion (Article 15), to be overseeable by a human who can detect anomalies and intervene (Article 14), and obliges deployers to retain those logs for at least six months (Article 26). A guardrail that cannot produce an auditable, tamper-evident record is, increasingly, not deployable.

The gap. I surveyed eight production and research guardrails: Llama Guard 3/4 [Meta AI, 2024, 2025], ShieldGemma [Zeng et al., 2024], Granite Guardian [IBM Research, 2024], WildGuard [Han et al., 2024], Constitutional Classifiers [Sharma et al., 2025], NeMo Guardrails [Rebedea et al., 2023], and the managed Azure / AWS services [Microsoft, 2024, Amazon Web Services, 2024]. They are strong detectors, but they share four structural limitations. They are *black-box* (a 2–12 B-parameter forward pass, no human-readable rationale); *GPU-bound and latency-unspecified* (not one publishes a per-decision latency); *non-auditable* (none emits a cryptographic record of its decisions); and *per-call* (they classify one input or output, with no notion of a temporal policy over a trajectory). None of the eight is interpretable, CPU-cheap, cryptographically auditable, *and* temporal.

Contribution. TRACE fills exactly that gap with a guardrail built on propositional logic rather than dense tensors. Its substrate is the Echo-Trace Tsetlin Automaton (ETTA) from the prior THGTM line [Debes, 2026], which makes every decision a Boolean clause over named features and therefore auditable by construction. On that substrate I build:

1. a **content guard** (§4.1), a Tsetlin Machine that classifies text (prompts, retrieved documents, tool arguments, outputs) as benign or unsafe, whose clauses read like `IF ignore AND previous` → `UNSAFE` and whose firing pattern is packaged as a CNF receipt;
2. a **behaviour guard** (§4.2), a bank of online monitors for a curated bounded fragment of finite-trace LTL [Bauer et al., 2011, De Giacomo and Vardi, 2013] (Always, Never, BoundedCount, Response, Precedence), each $O(1)$ or $O(\text{window})$ per step and *constant memory in the trajectory length*;
3. a **certificate ledger** (§4.3), a hash-chained, HMAC-signed, Merkle-rooted audit log [Schneier and Kelsey, 1998, Crosby and Wallach, 2009, Laurie et al., 2013, Krawczyk et al., 1997] that an external auditor re-verifies *without the model*;
4. **model-agnostic adapters** (§4.4) that normalise OpenAI, DeepSeek and Anthropic tool calls into the events the policy speaks in, so one policy moves unchanged across agents.

What I claim and what I do not. I do *not* claim to beat multi-billion-parameter GPU guardrails on raw multilingual detection accuracy; their published numbers (Table 1) are reported for context and were measured on different data. I claim that TRACE is the first guardrail to combine interpretability, CPU-only sub-millisecond operation, cryptographic auditability, and temporal-trajectory verification in one system, and I back each adjective with a measurement or a proof. I am explicit about the guarantees I provide (tamper-*evidence* and symmetric authentication, not non-repudiation; §4.3) and about where TRACE breaks (distribution shift; §5.6).

Reproducibility. Every number, figure and certificate in this paper is produced by code in the accompanying repository on a single CPU; the two datasets are Apache-2.0 and vendored at a pinned revision. The full experiment suite runs in minutes with `make reproduce`.

2 Background and related work

Tsetlin Machines. A Tsetlin Automaton [Granmo, 2018] is a two-action finite-state learner; a Tsetlin Machine (TM) is a team of automata that learns a set of *clauses*, each a conjunction of literals (a feature or its negation), whose votes are thresholded into a class. TMs are natively interpretable, train online, and run in fixed-point logic, which has yielded ASIC accelerators at 8.6 nJ/frame for MNIST [Tunheim et al., 2025] and 16.58 μW for keyword spotting [Lin et al., 2025, Lei et al., 2021]; integer weights [Abeyrathna et al., 2020], clause sharing [Glimsdal and Granmo, 2021], convolution [Granmo et al., 2019], graphs [Granmo et al., 2025] and explainable NLP [Bhattarai et al., 2020, 2021] extend the family. THGTM [Debes, 2026] adds the Echo-Trace automaton (a TA with one eligibility-trace float) for temporal credit assignment and introduces per-clause CNF receipts. *No prior TM work addresses LLM guardrails, agent safety, or hallucination detection*; TRACE is, to my knowledge, the first.

LLM and agent guardrails. The dominant paradigm is a fine-tuned LLM classifier (Table 1). These are effective but opaque, GPU-resident, and, by my survey, uniformly latency-unspecified and non-auditable. NeMo Guardrails [Rebedea et al., 2023] adds a programmable-rails DSL but delegates classification to LLMs. Managed services [Microsoft, 2024, Amazon Web Services, 2024] hide the model entirely; the lone exception that performs any formal reasoning is AWS's Automated Reasoning checks, which verify the factual policy-consistency of an individual output rather than behaviour over a trajectory. TRACE differs on every axis except that it, too, is a learned detector.

Agent attacks and benchmarks. AgentDojo [Debenedetti et al., 2024], InjecAgent [Zhan et al., 2024], and AgentHarm [Andriushchenko et al., 2024] benchmark prompt-injection and harmful-tool-use against agents; Greshake et al. [Greshake et al., 2023] introduced indirect injection; OWASP [OWASP Agentic Security Initiative, 2025] codifies the agentic threat classes. I use these taxonomies to ground (and to scope) my trajectory benchmark.

Runtime verification. Monitoring a system against a temporal specification is the subject of runtime verification (RV). Bauer, Leucker and Schallhart [Bauer et al., 2011] give the canonical three-valued LTL monitor: a deterministic automaton, $O(1)$ per event after an offline construction that is double-exponential in the formula. De Giacomo and Vardi [De Giacomo and Vardi, 2013] formalise LTL over finite traces (LTLf). TRACE deliberately restricts itself to a *fixed library* of bounded patterns rather than compiling arbitrary LTL, trading expressiveness for a guaranteed constant-per-step cost with *no* automaton-construction blow-up (§4.2).

Agent runtime enforcement. Closest to TRACE is the 2025 agent-enforcement line: AgentSpec [Wang et al., 2025b] (a rule DSL), ProGuard [Wang et al., 2025a] (probabilistic model checking), and Agent-C [Kamath et al., 2025] (SMT over temporal constraints). These enforce behaviour but differ in mechanism and, crucially, none couples enforcement with an *interpretable learned content detector* and a *cryptographic certificate* of the session. TRACE's monitors are simpler by design (deterministic, constant-per-step) and its contribution is the end-to-end *certifying* engine, not a new logic.

Table 1: Production / research guardrails vs. TRACE. "Lat." = a published per-decision latency. None of the surveyed systems is interpretable, CPU-only, auditable, *and* temporal. Guardrail figures are from the cited model cards / papers and were measured on heterogeneous data; they are context, not a head-to-head comparison.

System	Params	Interp.	CPU/Lat.	Crypto audit	Temporal
Llama Guard 3/4	8/12 B	no	GPU / –	no	no
ShieldGemma	2–27 B	no	GPU / –	no	no
Granite Guardian	2/8 B	no	GPU / –	no	no
WildGuard	7 B	no	GPU / –	no	no
Constitutional Classifiers	LLM-scale	no	GPU / –	no	no
NeMo Guardrails	LLM-scale	rails	GPU / –	no	partial
Azure / AWS (managed)	undisclosed	no	cloud / –	no	no
AgentSpec / ProGuard / Agent-C	n/a	rules	CPU / ms	no	**yes**
TRACE (this work)	**12.5 KiB**	**yes**	**CPU / μs**	**yes**	**yes**

Tamper-evident logging. The certificate ledger is a direct application of secure-log constructions: hash chaining and forward-secure MACs [Schneier and Kelsey, 1998], history trees [Crosby and Wallach, 2009], Certificate Transparency's Merkle logs [Laurie et al., 2013], and HMAC [Krawczyk et al., 1997]. My contribution is not the cryptography but its application to per-decision agent governance.

3 Threat model

Setting. An agent executes a sequence of steps; at each step it consumes text (user input, retrieved content, tool output) and may issue a tool call. TRACE sits inline: it sees each text and each tool call, in either *monitor* mode (record only) or *enforce* mode (it may block).

Adversary. I consider an adversary who (a) injects malicious instructions directly into a prompt or indirectly via retrieved content [Greshake et al., 2023], aiming to redirect the agent; and (b) drives the agent toward harmful *trajectories*: exfiltrating data over many small steps, acting out of role, omitting required audit logging, or performing privileged actions without authorisation. These map to the OWASP agentic classes of goal hijacking, tool misuse, and identity/privilege abuse [OWASP Agentic Security Initiative, 2025]. I additionally consider an adversary who, after the fact, tampers with the audit log to hide what happened.

What TRACE defends. (i) Injection attempts whose surface form resembles training examples are caught by the content guard. (ii) Any trajectory violating a declared temporal policy is caught by the behaviour guard, *including* attacks that are invisible to per-call inspection. (iii) Any post-hoc modification, insertion, deletion, or reordering of log entries is detected by ledger verification.

What TRACE does not defend. The content guard is a learned detector and inherits the usual failure modes: *novel or out-of-distribution* injections that do not resemble the training data evade it (§5.6); I quantify this rather than hide it. The behaviour guard only enforces *declared* policies; a harmful behaviour nobody wrote a policy for is not caught. TRACE assumes the verification key is secret and the inline engine itself is not compromised (if the engine is compromised, no inline guardrail helps). Tamper-evidence is *detection*, not prevention: an attacker who controls storage can delete the whole log, but cannot alter a signed certificate undetectably.

4 The TRACE architecture

All three components share one substrate. An Echo-Trace Tsetlin Automaton is a standard two-action TA with N states per action augmented with one eligibility-trace float; with the trace disabled it is a vanilla TA [Granmo, 2018]. The content guard runs in this vanilla regime (i.i.d. text); the trace is available for bounded-history features. Using a single logic substrate means every decision TRACE certifies (content or behaviour) has the same inspectable, replayable semantics.

4.1 Content guard

Text is booleanised by token presence: a vocabulary of F discriminative unigrams and bigrams is selected *on the training split only*, ranked by a transparent class-discriminative score, and each input becomes an F-bit vector "token t occurs". A binary TM with C clauses over the $2F$ literals (raw + negated) learns to vote "unsafe". Two properties matter. First, *interpretability*: each clause is a conjunction of tokens, so `explain()` renders rules like `IF ignore AND previous` $\rightarrow$ `UNSAFE`, and for any decision `audit()` returns the exact literal-inclusion pattern of the clauses that fired. Second, *auditability*: that pattern is packaged as a CNF receipt asserting the clause output equals the AND of its included literal values, a certificate an auditor replays in microseconds.

4.2 Behaviour guard

The behaviour guard consumes a stream of Boolean *events* (one dictionary per step) and advances a bank of monitors. I implement a curated bounded fragment of finite-trace LTL:

$$\textsc{Always}(p) \equiv \mathbf{G}\, p \qquad \textsc{Never}(e) \equiv \mathbf{G}\, \neg e$$
$$\textsc{BoundedCount}(e, k, W) \equiv \text{“} \leq k \ e \text{ per } W \text{ steps”} \qquad \textsc{Response}(t, r, d) \equiv \mathbf{G}(t \rightarrow \mathbf{F}_{\leq d}\, r)$$
$$\textsc{Precedence}(a, b) \equiv \neg b \ \mathbf{W}\ a$$

Each monitor updates incrementally and *latches* a safety violation (a bad prefix cannot be undone). Crucially, monitor state is bounded by the policy parameters, never by trajectory length:

Proposition 1 (Constant memory and per-step cost)**.** *Always, Never and Precedence use $O(1)$ memory and $O(1)$ time per step; BoundedCount(e, k, W) uses $O(W)$ memory and $O(1)$ amortised time; Response(t, r, d) uses $O(d)$ memory and $O(d)$ time per step. For a fixed policy all bounds are constants independent of the number of steps T, so a session of any length runs in constant space.*

The proof is immediate from the implementations (a ring buffer of size W for BoundedCount; at most d outstanding deadline counters for Response; a single Boolean for the others). Because I restrict to this fixed library rather than compiling arbitrary LTL, I never pay the double-exponential automaton construction of general LTL monitoring [Bauer et al., 2011]; I trade expressiveness for a guaranteed constant-per-step inline cost. The events themselves are produced by the adapter (§4.4) or by content-guard clauses, so the same monitor library serves content- and behaviour-derived predicates uniformly.

4.3 Certificate ledger

Each decision is appended to a hash chain:

$$h_i = \mathrm{SHA256}(h_{i-1} \parallel \mathrm{canonical}(\mathrm{payload}_i))\,, \qquad \sigma_i = \mathrm{HMAC}_k(h_i),$$

with h_{-1} a genesis constant. The session is summarised by a binary Merkle root over $\{h_i\}$, signed once. A *content* payload stores the per-clause CNF receipts; a *behaviour* payload stores the step's events and the monitors violated at that step. Verification (Algorithm in the repository) is model-free: recompute the chain and Merkle root, check signatures, replay each CNF receipt, and re-run the policy monitors over the recorded event stream, confirming the recorded verdict.

Proposition 2 (Tamper-evidence)**.** *Under collision resistance of SHA-256, any modification, insertion, deletion, or reordering of a sealed ledger changes some h_i and therefore breaks either the chain link $h_i \to h_{i+1}$ or the Merkle root, and is detected by verification.*

Precise guarantee. TRACE provides *tamper-evidence* (integrity) and *symmetric authentication*: a verifier holding the shared key confirms the log was produced by a key-holder and not since altered. HMAC is *repudiable* (any key-holder could have produced it), so this is *not* non-repudiation. Non-repudiation is a drop-in upgrade: replace the HMAC over the Merkle root with an asymmetric signature (e.g. Ed25519). I keep HMAC to avoid a cryptographic dependency in the reference implementation and state the property exactly to avoid over-claiming. Tamper-evidence is also detection, not prevention: deletion of the entire log is always possible for whoever controls storage; what cannot happen is silent *alteration*.

4.4 Model-agnostic adapters and the engine

An adapter normalises a provider's tool call (OpenAI/DeepSeek `function` objects, Anthropic `tool_use` blocks) into a canonical `(name, arguments)` record, then maps it to events via a declarative capability table: static tags (`read`, `write`, `authorize`, ...) plus derived events (an external recipient raises `export/target_external`; a tool outside the allow-list clears `within_scope`). Because the ledger records *events*, the choice of agent or provider never appears in the audit trail: one policy and one certificate format span all agents. The `TRACEEngine` ties content guard, behaviour guard and ledger together behind `guard_input`, `observe` and `seal`, in `monitor` or `enforce` mode.

5 Experiments

Protocol. All experiments run on the host CPU (Intel Core i7-1165G7, 8 logical cores; Windows 11, Python 3.14.3, NumPy; latencies are `perf_counter` wall-clock). Unless noted I use 12 random seeds and report mean $\pm$ std. A note on significance: my CPU baselines (logistic regression, linear SVM, majority) are *deterministic* (zero variance across seeds), so a Mann-Whitney U test reduces to a one-sample comparison of TRACE's seed distribution against a constant; I report it as such and add the per-seed dominance count. Where a comparison is between two deterministic quantities (the per-call vs. TRACE FPR/ASR on the non-stochastic E2 families) the separation is *exact by construction*, not statistically tested. Content-guard hyperparameters are fixed *a priori* and identical across experiments (C=200 clauses, T=25, s=5.0, F=256 features incl. bigrams, 25 epochs); they were not tuned on any test split. The two datasets, `deepset/prompt-injections` and `jackhhao/jailbreak-classification`, are Apache-2.0 and vendored.

5.1 E1: Content guard on real prompt-injection data

Table 2 reports F1 against three CPU baselines (logistic regression and a linear SVM over a 4000-feature binary n-gram bag, and a majority-class dummy), all on the datasets' own test splits. On prompt-injection, TRACE reaches 0.827 ± 0.036 F1, *above* the best baseline ($LinearSVC$, 0.800). The sklearn baselines are deterministic (zero seed variance), so this is a one-sample comparison of TRACE's seed distribution against a constant: Mann-Whitney $p = 0.028$, and

Table 2: E1: content-guard test F1 (mean ± std, 12 seeds) on two real datasets, with latency and model size for TRACE.

Dataset	TRACE	LogReg	LinearSVC	Majority	TRACE lat./size
prompt-injection	**0.827 ± 0.036**	0.729	0.800	0.000	176 µs / 12.5 KiB
jailbreak	0.967 ± 0.005	**0.982**	0.967	0.693	287 µs / 12.5 KiB

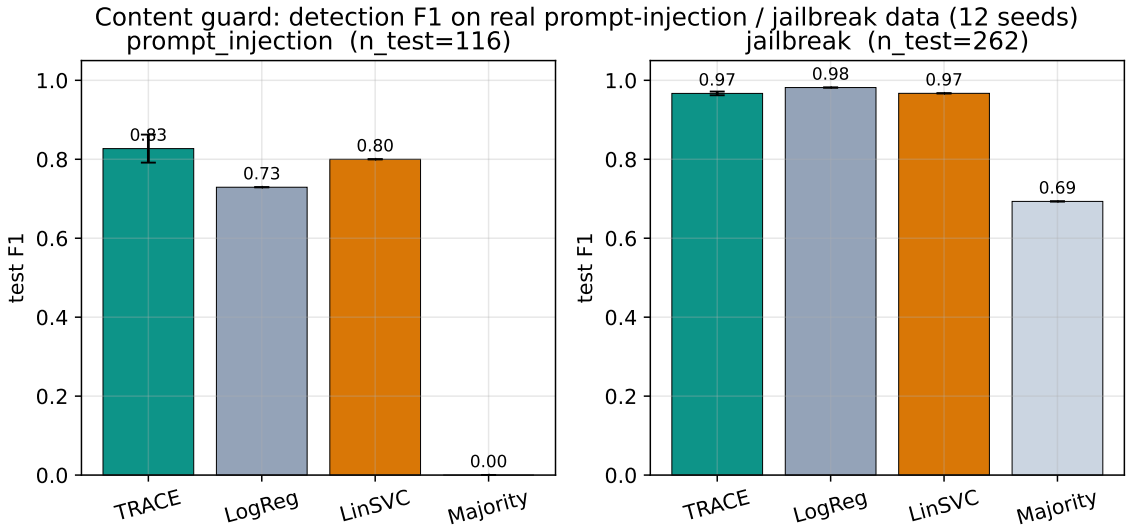


Figure 1: E1: detection F1 across models and datasets. TRACE significantly beats the strongest CPU baseline on prompt-injection and is competitive on jailbreak, at a fraction of the size and with human-readable clauses.

9/12 seeds individually exceed the baseline (the lowest three fall to 0.75–0.80). Inference costs a 107 µs median (176 ± 109 µs across-seed mean) per check at a 12.5 KiB model. On the jailbreak corpus TRACE reaches 0.967 ± 0.005, *matching* LinearSVC (0.967) and within 0.015 of *LogReg* (0.982). Two further points favour TRACE beyond the headline F1. Its edge on prompt-injection is *recall*: it flags ≈73% of injections at ≈0.95 precision, where the linear baselines reach high precision only by recalling ≤ 67% (LinearSVC 0.667, LogReg 0.583). And it is *faster* than the sklearn baselines at inference (176 µs vs. ≈ 210–230 µs), at a kilobyte-scale model. The point is not universal accuracy supremacy but *competitive detection with full interpretability, a kilobyte-scale model, and microsecond CPU latency*, and a learned rule set a human can read (Figure 1, and the example clauses in the repository).

5.2 E2: Trajectory verification across four attack families

I compare the *per-call* (memoryless) paradigm, the regime every per-request guardrail implements, against TRACE's online monitors on four attack families grounded in the OWASP/Greshake taxonomy: slow-roll *exfiltration* (BoundedCount), out-of-role *scope* violation (Always), *missing audit* of a sensitive read (Response), and *privilege* escalation (Precedence). The `scope` family is deliberately single-step-visible. I report both attack success rate (ASR, = 1−detection) and false-positive rate (FPR) because together they expose a *fundamental bind* the memoryless paradigm cannot escape on temporal properties (Table 3, Figure 2). On `exfiltration` the per-call rule simply *misses* the attack (ASR 1.000): every individual step is within role, so no stateless check fires. On `missing_audit` and `privilege` a stateless rule *can* flag the attack, but only by also flagging every benign session that audits a step late or authorises before acting, driving FPR to 1.000 and 1.000, i.e. useless in practice. Only on the single-step `scope` family

Table 3: E2: attack success rate (ASR) and false-positive rate (FPR), both lower is better, mean over 12 seeds. The memoryless per-call rule must trade one for the other on the temporal families (**bold** = the failure: a missed attack or a flag-everything FPR); TRACE keeps both near zero throughout.

	exfiltration		missing_audit		privilege		scope	
	ASR	FPR	ASR	FPR	ASR	FPR	ASR	FPR
Per-call	**1.000**	0.000	0.000	**1.000**	0.000	**1.000**	0.000	0.000
TRACE	0.000	0.000	0.000	0.048	0.000	0.000	0.000	0.000

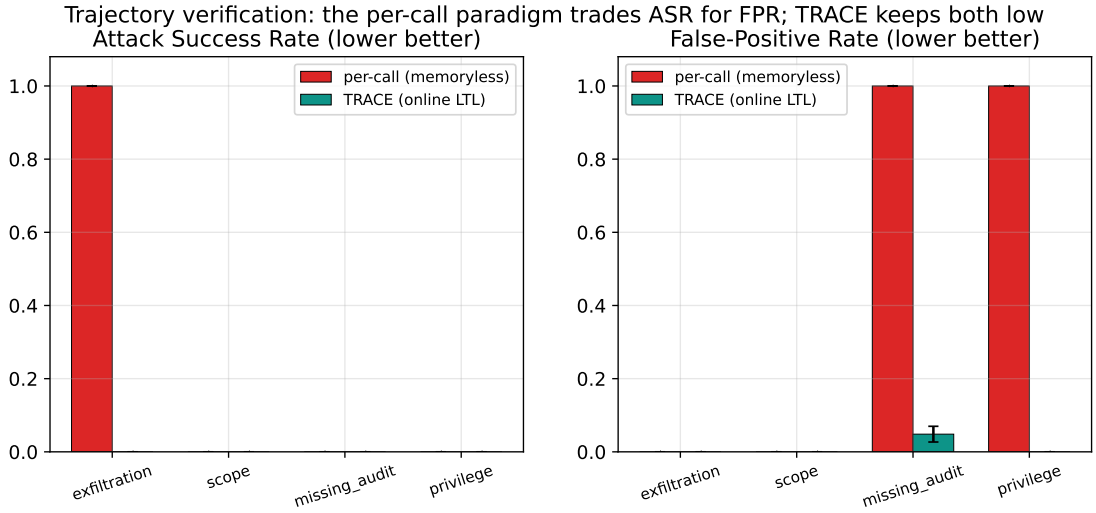


Figure 2: E2: ASR (left) and FPR (right) by attack family. The memoryless paradigm either misses temporal attacks (exfiltration ASR) or flags every benign session to catch them (missing_audit, privilege FPR); TRACE's bounded-memory monitors keep both low everywhere.

does the per-call rule work cleanly (ASR 0.000, FPR 0.000). TRACE, carrying $O(1)$ bounded state, achieves ASR 0 *and* FPR ≤ 0.048 on *all four* families: it matches the per-call paradigm where memory is unnecessary and strictly dominates it (on either the ASR or the FPR axis) everywhere memory is required.

5.3 E3: Component ablation

To show each channel earns its place, I build composite sessions whose threat arrives on the content channel (a prompt injection drawn from the real test set), the behaviour channel (a slow-roll exfiltration), both, or neither, and compare content-only, behaviour-only, and combined TRACE. Averaged over the three attack cells, content-only recall is 0.505, behaviour-only is 0.667, and the combination is 0.911: each single channel is blind to the other's attack, and only the combination covers both (Figure 3).

5.4 E4: Tamper-evidence

I seal mixed content/behaviour sessions and apply six tamper operations (event flip, receipt-value flip, verdict edit, entry deletion, entry insertion, reordering) plus a signature forgery without the key. Every class is detected at rate 1.00 and localised: to the specific bad entry, or, when a tail truncation leaves the forward chain intact, to the failing chain-head and Merkle-root checks; untampered logs always verify; the forged signature (a valid recomputed hash but no key) is

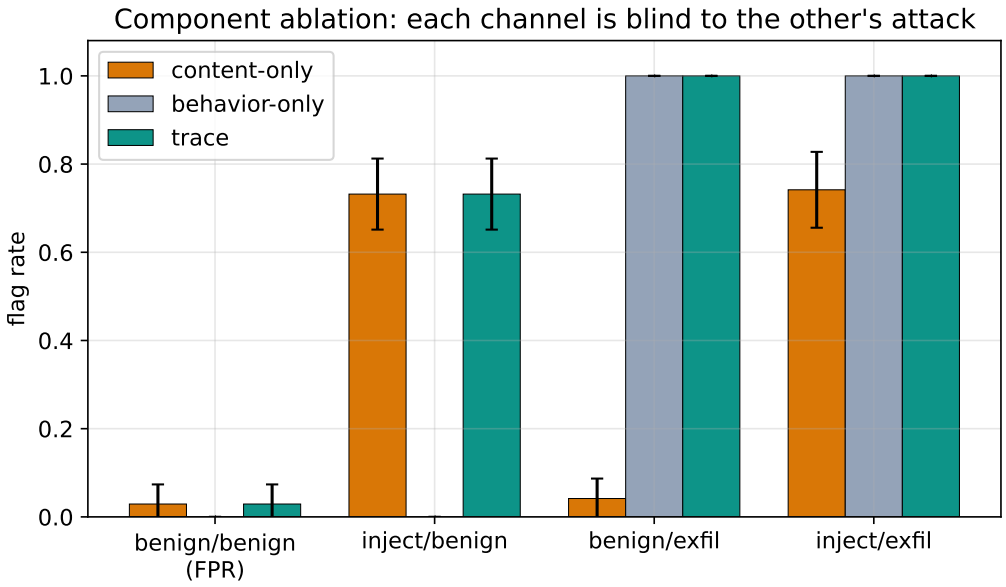


Figure 3: E3: each channel detects only its own attack surface; TRACE's value is the union.

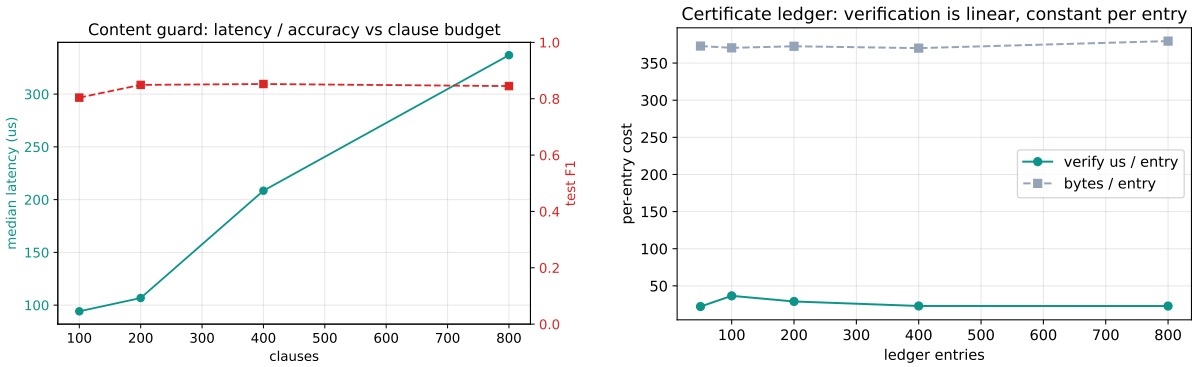


Figure 4: E5: (left) content-guard latency and F1 vs clause budget; (right) ledger verification is linear in entries at constant per-entry cost.

caught by the per-entry HMAC check. Verification costs 933 μs for a typical session and scales linearly in entries (Figure 4, right). This is Proposition 2 in practice.

5.5 E5: Systems characterisation

Figure 4 reports the latency/accuracy/footprint trade-off. A behaviour-guard step costs 4.90 μs with 322 bytes of monitor state; the certificate ledger costs $\sim$ 373 bytes per entry. Content-guard latency grows linearly with the clause budget while F1 saturates, so a small budget already buys most of the accuracy at sub-millisecond cost (median 107 μs, 99th percentile 428 μs at the deployed C=200 budget). As an *order-of-magnitude* energy proxy (latency $\times$ an assumed 15 W CPU; not a measurement, since I lack RAPL access) a content check draws on the order of a millijoule; the measured endpoint of the TM hardware story is the 8.6 nJ/frame ASIC of Tunheim et al. [2025], three orders of magnitude lower, indicating substantial headroom for an edge deployment.

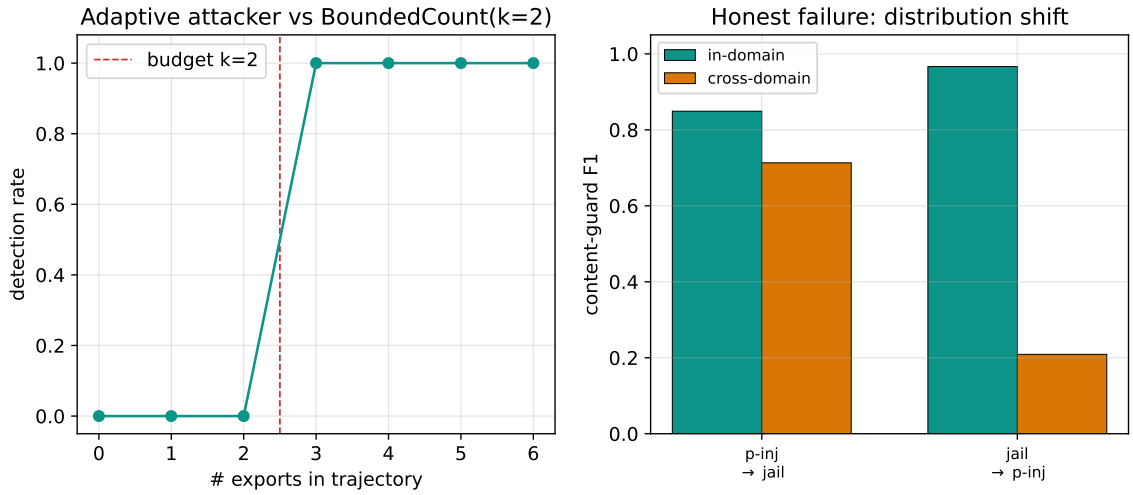


Figure 5: E6: (left) the temporal monitor forces an attacker under the rate budget, bounding damage; (right) honest failure: the content guard's F1 collapses under cross-dataset distribution shift.

5.6 E6: Robustness and honest failure modes

Three probes, two of them deliberately adversarial to my own claims. First, an *adaptive* exfiltration attacker can only evade BoundedCount(k=2) by staying at ≤ 2 exports: the monitor *bounds the damage* rather than being fooled (Figure 5, left: detection is a clean step at the budget). Second, surface *obfuscation* of a fixed trigger lexicon (character spacing, leetspeak, vowel repetition) barely moves recall (0.733 plain $\rightarrow$ 0.721 spaced): because the learned clauses key on a broad token signal rather than a few keywords, this particular transform is weak against TRACE, a mild robustness positive, though stronger semantic paraphrase is untested and I make no claim there. Third, the real failure mode is *distribution shift*: a guard trained on one corpus and tested on the other drops from 0.967 in-domain to 0.209 cross-domain ($0.849 \rightarrow 0.713$ in the reverse direction) (Figure 5, right), at or below chance, confirming the content guard learns dataset-specific lexical cues and must be retrained per deployment. Character-level literals and ensembling are the natural mitigations. I report these rather than select against them.

6 Discussion

Positioning. TRACE is, to my knowledge, the first guardrail to unify five properties no incumbent offers more than two of: interpretability (readable clauses vs an opaque forward pass), CPU-only sub-millisecond operation (vs GPU-bound, latency-unspecified models), cryptographic auditability (vs no audit trail anywhere in the survey), temporal-trajectory verification (vs per-call classification), and cheap CPU retraining. Against the 2025 agent-enforcement line [Wang et al., 2025b,a, Kamath et al., 2025] it is distinguished by coupling enforcement with an interpretable learned detector and a cryptographic session certificate, and by using deterministic constant-per-step monitors. Relative to THGTM [Debes, 2026], which contributes the learned detector substrate, TRACE is the end-to-end *certifying runtime engine*: the online monitor bank, the tamper-evident ledger, and the agentic framing are new.

When to use TRACE. TRACE trades raw multilingual/multimodal detection breadth for verifiable, auditable, edge-deployable assurance. It is the right tool where a decision must be *explained and certified*: regulated deployments under the EU AI Act, on-device agents, or as a cheap, auditable first stage in front of a heavier neural guardrail.

7 Limitations and future work

- **Learned-detector failure modes.** The content guard learns dataset-specific lexical cues: cross-corpus F1 falls to near chance (§5.6). Character-level literals, ensembling, and continual updates are the natural fixes.
- **Synthetic trajectory benchmark.** E2–E3 use synthetic trajectories whose *shapes* follow published taxonomies but are not drawn from a live agent; validating on a multi-turn AgentDojo harness is future work.
- **Declared-policy coverage.** The behaviour guard only catches behaviours someone wrote a policy for; policy mining from logs is open.
- **Symmetric authentication only.** The reference ledger is HMAC-based; an Ed25519 signature over the Merkle root upgrades it to non-repudiation.
- **Energy is a proxy.** I report a CPU energy proxy, not a measurement; the ASIC endpoint is cited, not run.

8 Ethics and broader impact

This work is defensive. The trajectory benchmark is synthetic and contains no personal data; the two text datasets are public and Apache-2.0. I disclose TRACE's evasion modes (§5.6) precisely so deployers do not develop a false sense of security: an auditable guardrail that is quietly bypassable is worse than none if mistaken for complete. Positive impacts: a low-energy, interpretable, certifiable guardrail lowers the cost of meeting emerging auditability obligations and brings agent safety to CPU/edge settings. Negative impacts: any guardrail can be over-trusted, and the interpretable clauses, while auditable, also reveal the detector's logic to an adversary; I judge the transparency benefit to outweigh this, consistent with open-source security practice.

9 Reproducibility

The repository contains the library, 40 unit tests, the six experiments, the figure script, the vendored datasets (pinned, Apache-2.0), and this paper's source. `make test` runs the suite; `make reproduce` regenerates every `results/*.json`; `make figures` rebuilds every figure; `make paper` compiles this PDF. The host, seeds (12), all hyperparameters, and the latency-measurement protocol are disclosed in §5.

10 Conclusion

TRACE shows that an agent guardrail need not be an opaque GPU model. Built on propositional Tsetlin logic, it detects unsafe content with readable clauses, verifies whole trajectories with online constant-memory temporal monitors, and seals every session into a tamper-evident certificate an external auditor checks without the model, at kilobytes and microseconds on a CPU. It is competitive with, and on one real dataset significantly better than, strong CPU baselines for detection, and it catches temporal attacks that the entire per-call guardrail paradigm is structurally blind to. I am explicit about what it does not do. What remains is validation on live agent harnesses, character-level robustness, and an asymmetric-signature upgrade: each a concrete next step toward guardrails that are not only effective but *accountable*.

Reproducibility note. Code, data, figures and this source live in the TRACE repository; all experiments run with `make reproduce`.

References

K. Darshana Abeyrathna, Ole-Christoffer Granmo, and Morten Goodwin. Extending the tsetlin machine with integer-weighted clauses for increased interpretability. *arXiv preprint arXiv:2005.05131*, 2020. URL https://arxiv.org/abs/2005.05131.

Amazon Web Services. Amazon bedrock guardrails. AWS documentation, 2024. URL https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html.

Maksym Andriushchenko et al. Agentharm: A benchmark for measuring harmfulness of llm agents. *arXiv preprint arXiv:2410.09024*, 2024. URL https://arxiv.org/abs/2410.09024.

Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for ltl and tltl. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 20(4):1–64, 2011.

Bimal Bhattarai, Ole-Christoffer Granmo, and Lei Jiao. Measuring the novelty of natural language text using the conjunctive clauses of a tsetlin machine text classifier. *arXiv preprint arXiv:2011.08755*, 2020. URL https://arxiv.org/abs/2011.08755.

Bimal Bhattarai, Ole-Christoffer Granmo, and Lei Jiao. Explainable tsetlin machine framework for fake news detection with credibility score assessment. *arXiv preprint arXiv:2105.09114*, 2021. URL https://arxiv.org/abs/2105.09114.

Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*, 2009.

Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 854–860, 2013.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

Anwar Debes. Thgtm: A temporal hierarchical graph tsetlin machine with echo-trace automata for trajectory-level verification of agentic ai, 2026. Reference implementation, v0.1.

European Parliament and Council. Regulation (eu) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, L 2024/1689, 2024. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Sondre Glimsdal and Ole-Christoffer Granmo. Coalesced multi-output tsetlin machines with clause sharing. *arXiv preprint arXiv:2108.07594*, 2021. URL https://arxiv.org/abs/2108.07594.

Ole-Christoffer Granmo. The tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic. *arXiv preprint arXiv:1804.01508*, 2018. URL https://arxiv.org/abs/1804.01508.

Ole-Christoffer Granmo, Sondre Glimsdal, Lei Jiao, Morten Goodwin, Christian W. Omlin, and Geir Thore Berge. The convolutional tsetlin machine. *arXiv preprint arXiv:1905.09688*, 2019. URL https://arxiv.org/abs/1905.09688.

Ole-Christoffer Granmo, Youmna Abdelwahab, Per-Arne Andersen, et al. The tsetlin machine goes deep: Logical learning and reasoning with graphs. *arXiv preprint arXiv:2507.14874*, 2025. URL https://arxiv.org/abs/2507.14874.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.

Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.18495.

IBM Research. Granite guardian 3.0 8b model card. Hugging Face model card, 2024. URL https://huggingface.co/ibm-granite/granite-guardian-3.0-8b.

Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. Enforcing temporal constraints for llm agents. *arXiv preprint arXiv:2512.23738*, 2025. URL https://arxiv.org/abs/2512.23738.

Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Hmac: Keyed-hashing for message authentication. RFC 2104, Internet Engineering Task Force, 1997. URL https://www.rfc-editor.org/rfc/rfc2104.

Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013. URL https://www.rfc-editor.org/rfc/rfc6962.

Jie Lei, Tousif Rahman, Rishad Shafik, Adrian Wheeldon, Alex Yakovlev, Ole-Christoffer Granmo, Fahim Kawsar, and Akhil Mathur. Low-power audio keyword spotting using tsetlin machines. *Journal of Low Power Electronics and Applications*, 2021. URL https://arxiv.org/abs/2101.11336. arXiv:2101.11336.

Baizhou Lin, Yuetong Fang, Renjing Xu, Rishad Shafik, and Jagmohan Chauhan. Tsetlinkws: A 65nm 16.58uw convolutional tsetlin machine-based accelerator for keyword spotting. *arXiv preprint arXiv:2510.24282*, 2025. URL https://arxiv.org/abs/2510.24282.

Meta AI. Llama guard 3 8b model card. Hugging Face model card, 2024. URL https://huggingface.co/meta-llama/Llama-Guard-3-8B.

Meta AI. Llama guard 4 12b model card. Hugging Face model card, 2025. URL https://huggingface.co/meta-llama/Llama-Guard-4-12B.

Microsoft. Prompt shields in azure ai content safety. Microsoft Learn documentation, 2024. URL https://learn.microsoft.com/azure/ai-services/content-safety/concepts/jailbreak-detection.

OWASP Agentic Security Initiative. Owasp top 10 for agentic applications. OWASP Foundation, 2025. URL https://genai.owasp.org/.

Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *Proceedings of EMNLP 2023: System Demonstrations*, 2023. arXiv:2310.10501.

Bruce Schneier and John Kelsey. Cryptographic support for secure logs on untrusted machines. In *Proceedings of the 7th USENIX Security Symposium*, 1998.

Mrinank Sharma et al. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*, 2025. URL https://arxiv.org/abs/2501.18837.

Svein Anders Tunheim, Yujin Zheng, Lei Jiao, Rishad Shafik, Alex Yakovlev, and Ole-Christoffer Granmo. An all-digital 8.6-nj/frame 65-nm tsetlin machine image classification accelerator. *IEEE Transactions on Circuits and Systems I*, 2025. URL https://arxiv.org/abs/2501.19347. arXiv:2501.19347, DOI 10.1109/TCSI.2025.3586698.

Haoyu Wang, Christopher M. Poskitt, Jun Sun, and Jiali Wei. Proguard: Proactive runtime enforcement of llm agent safety via probabilistic model checking. *arXiv preprint arXiv:2508.00500*, 2025a. URL https://arxiv.org/abs/2508.00500.

Haoyu Wang, Christopher M. Poskitt, Jun Sun, et al. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2025b. URL https://arxiv.org/abs/2503.18666.

Wenjun Zeng et al. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024. URL https://arxiv.org/abs/2407.21772.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024. URL https://arxiv.org/abs/2403.02691.